

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 15.

**Тема: «Разработка комплексных тестовых сценариев
(Unit + Integration + Contract)**

Язык программирования: Python 3.10+»

Выполнила: Потанина Н.А.,

Студентка группы: 24-ИС.

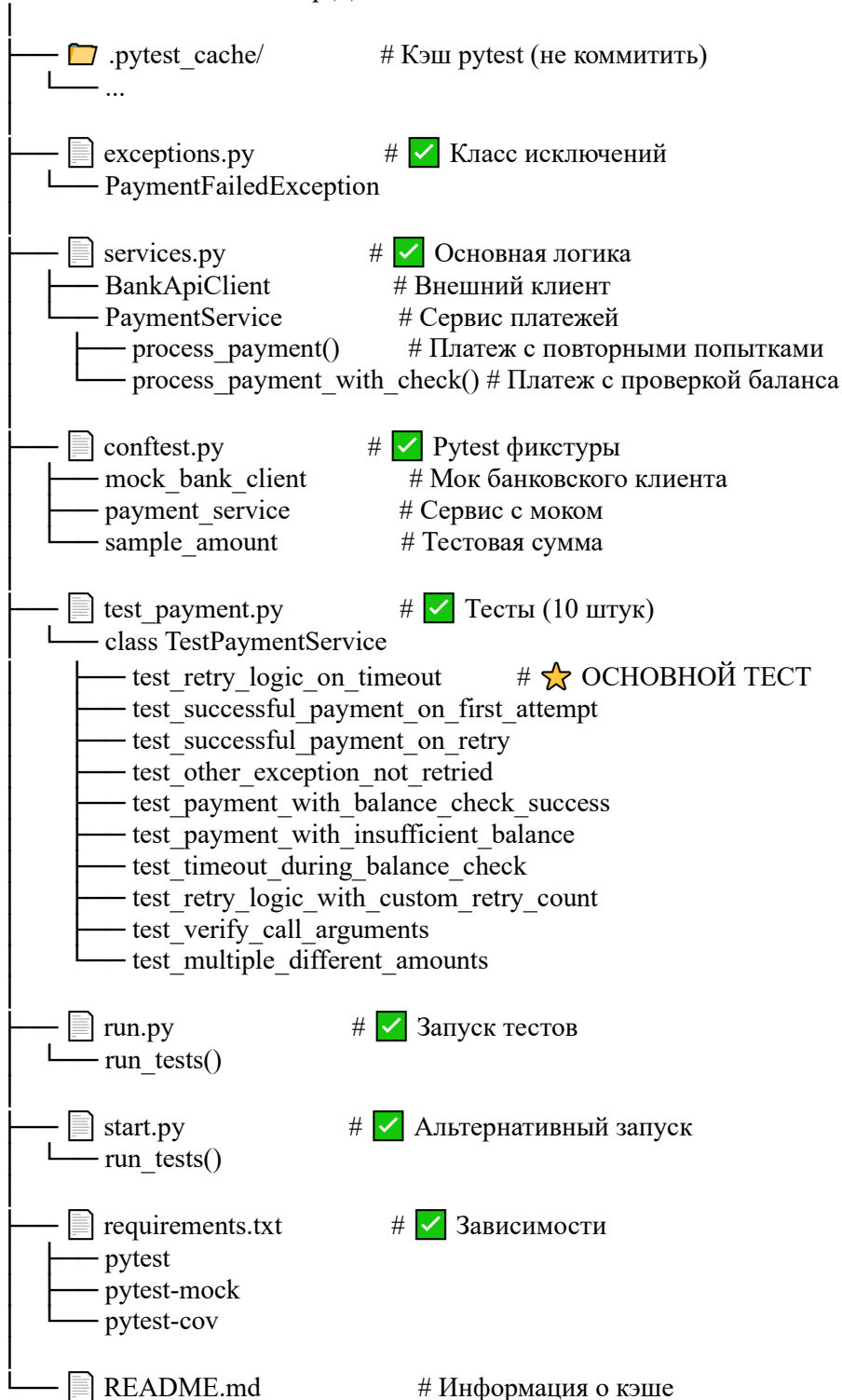
2026г.

Цель работы:

Разработать набор тестов (модульных, интеграционных и контрактных) для микросервиса «Система управления заказами и доставкой», обеспечивающих надежность модулей авторизации, обработки заказов и внешних интеграций (платежи, логистика).

Вариант 2. Модульное тестирование с моками (Mock)

C:/Users/Student237/Desktop/День 15/



Структура данных

```

class PaymentFailedException(Exception):
    def __init__(self, message="Payment processing failed"):
        self.message = message
        super().__init__(self.message)

```

1.exceptions.py

```

import time
import logging
from exceptions import PaymentFailedException

```

```

logger = logging.getLogger(__name__)

```

```

class BankApiClient:
    def charge(self, amount):
        raise NotImplementedError("Real implementation should make HTTP request")

    def get_balance(self, account_id):
        raise NotImplementedError("Real implementation should make HTTP request")

```

```

class PaymentService:
    def __init__(self, bank_client, max_retries=3, retry_delay=0.1):
        self.bank_client = bank_client
        self.max_retries = max_retries
        self.retry_delay = retry_delay

    def process_payment(self, amount):
        logger.info(f"Processing payment of ${amount:.2f}")

        for attempt in range(1, self.max_retries + 1):
            try:
                result = self.bank_client.charge(amount)

                return {
                    "status": "success",
                    "amount": amount,
                    "transaction_id": result.get("transaction_id"),
                    "attempts": attempt
                }

            except TimeoutError as e:
                logger.warning(f"Timeout on attempt {attempt}: {e}")

                if attempt < self.max_retries:
                    time.sleep(self.retry_delay)
                    continue

            except Exception as e:
                logger.error(f"Critical error during payment: {e}")
                raise PaymentFailedException(f"Critical error: {e}")

        logger.error(f"All {self.max_retries} payment attempts failed")
        raise PaymentFailedException("Bank unavailable - all retry attempts failed")

    def process_payment_with_check(self, amount, account_id):
        logger.info(f"Processing payment with balance check for account {account_id}")

```

```

try:
    balance = self.bank_client.get_balance(account_id)

    if balance < amount:
        raise PaymentFailedException(
            f'Insufficient funds: balance ${balance:.2f}, required ${amount:.2f}'
        )

    result = self.bank_client.charge(amount)

    return {
        "status": "success",
        "amount": amount,
        "transaction_id": result.get("transaction_id"),
        "balance_after": result.get("new_balance", balance - amount)
    }

except TimeoutError as e:
    logger.error(f'Timeout during payment processing: {e}')
    raise PaymentFailedException(f'Bank timeout: {e}')
except PaymentFailedException:
    raise
except Exception as e:
    logger.error(f'Unexpected error: {e}')
    raise PaymentFailedException(f'Payment processing error: {e}')

```

2. services.py

```

import pytest
from unittest.mock import Mock
from services import PaymentService

@pytest.fixture
def mock_bank_client():
    return Mock()

@pytest.fixture
def payment_service(mock_bank_client):
    return PaymentService(mock_bank_client, max_retries=3, retry_delay=0.01)

@pytest.fixture
def sample_amount():
    return 100.0

```

3. conftest.py

```

"""
Тесты для PaymentService (Вариант 2)
Модульное тестирование с моками (Mock)
"""

import pytest
from unittest.mock import call

from services import PaymentService
from exceptions import PaymentFailedException

class TestPaymentService:

    def test_retry_logic_on_timeout(self, mocker):

```

```

"""
ОСНОВНОЙ ТЕСТ: Проверка логики повторных попыток при TimeoutError.
"""

# Arrange
mock_client = mocker.Mock()
mock_client.charge.side_effect = TimeoutError("Connection lost")

payment_service = PaymentService(
    bank_client=mock_client,
    max_retries=3,
    retry_delay=0.001
)

# Act & Assert
with pytest.raises(PaymentFailedException) as exc_info:
    payment_service.process_payment(100.0)

assert "Bank unavailable" in str(exc_info.value)
assert mock_client.charge.call_count == 3
mock_client.charge.assert_called_with(100.0)

def test_successful_payment_on_first_attempt(self, mocker):
    """Тест успешного платежа с первой попытки."""
    # Arrange
    mock_client = mocker.Mock()
    mock_client.charge.return_value = {
        "transaction_id": "txn_12345",
        "status": "completed"
    }
    payment_service = PaymentService(mock_client)

    # Act
    result = payment_service.process_payment(100.0)

    # Assert
    assert mock_client.charge.call_count == 1
    mock_client.charge.assert_called_with(100.0)
    assert result["status"] == "success"
    assert result["amount"] == 100.0
    assert result["transaction_id"] == "txn_12345"
    assert result["attempts"] == 1

def test_successful_payment_on_retry(self, mocker):
    """Тест успешного платежа после неудачных попыток."""
    # Arrange
    mock_client = mocker.Mock()
    mock_client.charge.side_effect = [
        TimeoutError("Connection timeout"),
        TimeoutError("Connection timeout"),
        {"transaction_id": "txn_67890", "status": "completed"}
    ]
    payment_service = PaymentService(mock_client, max_retries=3, retry_delay=0.001)

    # Act
    result = payment_service.process_payment(150.0)

    # Assert

```

```

assert mock_client.charge.call_count == 3
expected_calls = [call(150.0)] * 3
mock_client.charge.assert_has_calls(expected_calls)
assert result["status"] == "success"
assert result["amount"] == 150.0
assert result["transaction_id"] == "txn_67890"
assert result["attempts"] == 3

def test_other_exception_not_retried(self, mocker):
    """Тест: другие исключения не вызывают повторных попыток."""
    # Arrange
    mock_client = mocker.Mock()
    mock_client.charge.side_effect = ValueError("Invalid amount")
    payment_service = PaymentService(mock_client)

    # Act & Assert
    with pytest.raises(PaymentFailedException, match="Critical error"):
        payment_service.process_payment(-100.0)

    assert mock_client.charge.call_count == 1
    mock_client.charge.assert_called_with(-100.0)

def test_payment_with_balance_check_success(self, mocker):
    """Тест успешного платежа с проверкой баланса."""
    # Arrange
    mock_client = mocker.Mock()
    mock_client.get_balance.return_value = 1000.0
    mock_client.charge.return_value = {
        "transaction_id": "txn_balance_001",
        "new_balance": 850.0
    }
    payment_service = PaymentService(mock_client)

    # Act
    result = payment_service.process_payment_with_check(150.0, "acc_123")

    # Assert
    mock_client.get_balance.assert_called_with("acc_123")
    mock_client.charge.assert_called_with(150.0)
    assert result["status"] == "success"
    assert result["amount"] == 150.0
    assert result["transaction_id"] == "txn_balance_001"
    assert result["balance_after"] == 850.0

def test_payment_with_insufficient_balance(self, mocker):
    """Тест: недостаток средств на счете."""
    # Arrange
    mock_client = mocker.Mock()
    mock_client.get_balance.return_value = 50.0
    payment_service = PaymentService(mock_client)

    # Act & Assert
    with pytest.raises(PaymentFailedException) as exc_info:
        payment_service.process_payment_with_check(100.0, "acc_123")

    assert "Insufficient funds" in str(exc_info.value)
    mock_client.get_balance.assert_called_with("acc_123")

```

```

mock_client.charge.assert_not_called()

def test_timeout_during_balance_check(self, mocker):
    """Тест: таймаут при проверке баланса."""
    # Arrange
    mock_client = mocker.Mock()
    mock_client.get_balance.side_effect = TimeoutError("Balance check timeout")
    payment_service = PaymentService(mock_client)

    # Act & Assert
    with pytest.raises(PaymentFailedException) as exc_info:
        payment_service.process_payment_with_check(100.0, "acc_123")

    assert "Bank timeout" in str(exc_info.value)
    mock_client.get_balance.assert_called_with("acc_123")
    mock_client.charge.assert_not_called()

def test_retry_logic_with_custom_retry_count(self, mocker):
    """Тест с кастомным количеством попыток."""
    # Arrange
    mock_client = mocker.Mock()
    mock_client.charge.side_effect = TimeoutError("Connection lost")
    payment_service = PaymentService(mock_client, max_retries=5, retry_delay=0.001)

    # Act & Assert
    with pytest.raises(PaymentFailedException):
        payment_service.process_payment(100.0)

    assert mock_client.charge.call_count == 5
    mock_client.charge.assert_called_with(100.0)

def test_verify_call_arguments(self, mocker):
    """Тест проверки аргументов вызова."""
    # Arrange
    mock_client = mocker.Mock()
    mock_client.charge.return_value = {"transaction_id": "txn_001"}
    payment_service = PaymentService(mock_client)

    # Act
    payment_service.process_payment(250.0)

    # Assert
    mock_client.charge.assert_called_with(250.0)
    calls = mock_client.charge.call_args_list
    assert len(calls) == 1
    assert calls[0].args == (250.0,)

def test_multiple_different_amounts(self, mocker):
    """Тест с разными суммами платежей."""
    # Arrange
    mock_client = mocker.Mock()
    mock_client.charge.side_effect = [
        {"transaction_id": "txn_001"},
        {"transaction_id": "txn_002"},
        {"transaction_id": "txn_003"}
    ]
    payment_service = PaymentService(mock_client)

```

```

# Act
result1 = payment_service.process_payment(100.0)
result2 = payment_service.process_payment(200.0)
result3 = payment_service.process_payment(300.0)

# Assert
assert mock_client.charge.call_count == 3
assert result1["amount"] == 100.0
assert result2["amount"] == 200.0
assert result3["amount"] == 300.0

expected_calls = [call(100.0), call(200.0), call(300.0)]
mock_client.charge.assert_has_calls(expected_calls)

```

4. test_payment.py

```

import subprocess
import sys

def run_tests():
    print("=" * 60)
    print("ЗАПУСК ТЕСТОВ ДЛЯ PAYMENT SERVICE (Вариант 2)")
    print("=" * 60)
    print()

    # Используем pytest напрямую
    cmd = [
        sys.executable,
        "-m",
        "pytest",
        "test_payment.py", # <--- Имя вашего файла с тестами
        "-v",
        "--tb=short"
    ]

    result = subprocess.run(cmd)

    print()
    print("=" * 60)
    if result.returncode == 0:
        print("✅ ВСЕ ТЕСТЫ ПРОЙДЕНЫ УСПЕШНО!")
    else:
        print("❌ ЕСТЬ ПРОБЛЕМЫ С ТЕСТАМИ")
    print("=" * 60)

    return result.returncode

if __name__ == "__main__":
    sys.exit(run_tests())

```

5. run.py

```

"""
Запуск тестов через командную строку
"""

import subprocess
import sys
import os

```

```

def run_tests():
    print("=" * 60)
    print("ЗАПУСК ТЕСТОВ ДЛЯ PAYMENT SERVICE (Вариант 2)")
    print("=" * 60)
    print()

    # Запускаем pytest через subprocess
    result = subprocess.run(
        [sys.executable, "-m", "pytest", "test_payment.py", "-v"],
        capture_output=True,
        text=True
    )

    # Выводим результат
    print(result.stdout)
    if result.stderr:
        print(result.stderr)

    print()
    print("=" * 60)
    if result.returncode == 0:
        print("✅ ВСЕ ТЕСТЫ ПРОЙДЕНЫ УСПЕШНО!")
    else:
        print("❌ ЕСТЬ ПРОБЛЕМЫ С ТЕСТАМИ")
    print("=" * 60)

    return result.returncode

if __name__ == "__main__":
    # Проверяем, что мы в правильной папке
    if not os.path.exists("test_payment.py"):
        print("❌ ОШИБКА: Файл test_payment.py не найден!")
        print(f"Текущая папка: {os.getcwd()}")
        sys.exit(1)

    sys.exit(run_tests())

```

6. start.py

pytest
 pytest-mock
 pytest-cov

7. requirements.txt

```
Командная строка
Microsoft Windows [Version 10.0.19045.6456]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\Student237>cd "C:/Users/Student237/Desktop/День 15"

C:\Users\Student237\Desktop\День 15>pip install -r requirements.txt
Requirement already satisfied: pytest in c:\users\student237\appdata\local\programs\python\python313\lib\site-packages (from -r requirements.txt (line 1)) (9.1.0)
Collecting pytest-mock (from -r requirements.txt (line 2))
  Downloading pytest_mock-3.15.1-py3-none-any.whl.metadata (3.9 kB)
Requirement already satisfied: pytest-cov in c:\users\student237\appdata\local\programs\python\python313\lib\site-packages (from -r requirements.txt (line 3)) (7.1.0)
Requirement already satisfied: colorama>=0.4 in c:\users\student237\appdata\local\programs\python\python313\lib\site-packages (from pytest->-r requirements.txt (line 1)) (0.4.6)
Requirement already satisfied: iniconfig>=1.0.1 in c:\users\student237\appdata\local\programs\python\python313\lib\site-packages (from pytest->-r requirements.txt (line 1)) (2.3.0)
Requirement already satisfied: packaging>=22 in c:\users\student237\appdata\local\programs\python\python313\lib\site-packages (from pytest->-r requirements.txt (line 1)) (26.2)
Requirement already satisfied: pluggy<2,>=1.5 in c:\users\student237\appdata\local\programs\python\python313\lib\site-packages (from pytest->-r requirements.txt (line 1)) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in c:\users\student237\appdata\local\programs\python\python313\lib\site-packages (from pytest->-r requirements.txt (line 1)) (2.20.0)
Requirement already satisfied: coverage>=7.10.6 in c:\users\student237\appdata\local\programs\python\python313\lib\site-packages (from coverage[toml]>=7.10.6->pytest-cov->-r requirements.txt (line 3)) (7.14.1)
Downloading pytest_mock-3.15.1-py3-none-any.whl (10 kB)
Installing collected packages: pytest-mock
Successfully installed pytest-mock-3.15.1

[notice] A new release of pip is available: 25.2 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\Student237\Desktop\День 15>pytest test_payment.py -v
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\Student237\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Student237\Desktop\День 15
plugins: hypothesis-6.155.3, cov-7.1.0, mock-3.15.1
collected 10 items

test_payment.py::TestPaymentService::test_retry_logic_on_timeout PASSED [ 10%]
test_payment.py::TestPaymentService::test_successful_payment_on_first_attempt PASSED [ 20%]
test_payment.py::TestPaymentService::test_successful_payment_on_retry PASSED [ 30%]
test_payment.py::TestPaymentService::test_other_exception_not_retried PASSED [ 40%]
test_payment.py::TestPaymentService::test_payment_with_balance_check_success PASSED [ 50%]
test_payment.py::TestPaymentService::test_payment_with_insufficient_balance PASSED [ 60%]
test_payment.py::TestPaymentService::test_timeout_during_balance_check PASSED [ 70%]
test_payment.py::TestPaymentService::test_retry_logic_with_custom_retry_count PASSED [ 80%]
test_payment.py::TestPaymentService::test_verify_call_arguments PASSED [ 90%]
test_payment.py::TestPaymentService::test_multiple_different_amounts PASSED [100%]
```

Рисунок 1 — Результат запуска тестов в командной строке

```
===== 10 passed in 0.45s =====

C:\Users\Student237\Desktop\День 15>pytest test_payment.py --cov=. --cov-report=term-missing
===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.1.0, pluggy-1.6.0
rootdir: C:\Users\Student237\Desktop\День 15
plugins: hypothesis-6.155.3, cov-7.1.0, mock-3.15.1
collected 10 items

test_payment.py ..... [100%]

===== tests coverage =====
coverage: platform win32, python 3.13.9-final-0

Name                Stmts   Miss  Cover   Missing
-----
conftest.py           12      3    75%    8, 13, 18
exceptions.py          4      0   100%
run.py                18     18     0%    1-33
services.py           46      5    89%    10, 13, 75-77
start.py              25     25     0%    5-44
test_payment.py       105      0   100%
TOTAL                 210     51    76%

===== 10 passed in 0.34s =====

C:\Users\Student237\Desktop\День 15>
```

Рисунок 2 — Отчет о покрытии кода (coverage)

```
===== RESTART: C:/Users/Student237/Desktop/День 15/start.py =====
=====
ЗАПУСК ТЕСТОВ ДЛЯ PAYMENT SERVICE (Вариант 2)
=====

===== test session starts =====
platform win32 -- Python 3.13.9, pytest-9.1.0, pluggy-1.6.0 -- C:\Users\Student237\AppData\Local\Programs\Python\Python313\pythonw.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Student237\Desktop\День 15
plugins: hypothesis-6.155.3, cov-7.1.0, mock-3.15.1
collecting ... collected 10 items

test_payment.py::TestPaymentService::test_retry_logic_on_timeout PASSED [ 10%]
test_payment.py::TestPaymentService::test_successful_payment_on_first_attempt PASSED [ 20%]
test_payment.py::TestPaymentService::test_successful_payment_on_retry PASSED [ 30%]
test_payment.py::TestPaymentService::test_other_exception_not_retried PASSED [ 40%]
test_payment.py::TestPaymentService::test_payment_with_balance_check_success PASSED [ 50%]
test_payment.py::TestPaymentService::test_payment_with_insufficient_balance PASSED [ 60%]
test_payment.py::TestPaymentService::test_timeout_during_balance_check PASSED [ 70%]
test_payment.py::TestPaymentService::test_retry_logic_with_custom_retry_count PASSED [ 80%]
test_payment.py::TestPaymentService::test_verify_call_arguments PASSED [ 90%]
test_payment.py::TestPaymentService::test_multiple_different_amounts PASSED [100%]

===== 10 passed in 0.85s =====

=====
✓ ВСЕ ТЕСТЫ ПРОЙДЕНЫ УСПЕШНО!
=====
```

Рисунок 3 — Успешное выполнение всех тестов

Ответы на контрольные вопросы:

Вопрос 1: Чем отличается pytest от unittest?

Ответ:

pytest — более современный, гибкий и простой в использовании

unittest — встроенный в Python, требует наследования от TestCase и методов assert*

У pytest проще синтаксис (просто assert), есть фикстуры, параметризация, плагины

pytest автоматически находит тесты по именам test_*.py / test_*

Вопрос 2: Что такое фикстуры (fixtures) в pytest и зачем они нужны?

Ответ:

Фикстуры — это функции с декоратором @pytest.fixture, которые подготавливают окружение для тестов (БД, клиенты, данные). Нужны для:

Переиспользования кода

Автоматического управления ресурсами (setup/teardown)

Инъекции зависимостей в тесты

Вопрос 3: Как в Python замочать внешний HTTP-запрос?

Ответ:

Способы:

1. **pytest-httpx** — httpx_mock.add_response()

- 2. **responses** — `responses.add()`
- 3. **unittest.mock** — замокать HTTP-
клиент: `mock_client.get.return_value = ...`
- 4. **httpretty** — перехватывает реальные HTTP-запросы

Вопрос 4: Что такое `@pytest.mark.parametrize`?

Ответ:

Декоратор для параметризации тестов. Позволяет запускать один тест с разными наборами данных:

```
python
@pytest.mark.parametrize("x,y,expected", [(1,2,3), (4,5,9)])
def test_add(x, y, expected):
    assert x + y == expected
```

Вопрос 5: Как проверить, что функция выбросила исключение с нужным текстом?

Ответ:

Использовать `pytest.raises` с параметром `match`:

```
python
with pytest.raises(ValueError, match="неверное значение"):
    function_bad(-1)
```

Вопрос 6: В чем разница между `mock.Mock()` и `mock.patch()`?

Ответ:

`Mock()` — создает объект-заглушку, который можно настроить (`return_value`, `side_effect`)

`patch()` — заменяет реальный объект на мок в определенном контексте (декоратор/менеджер контекста)

```
python
mock_obj = Mock()           # создание мока
with patch('module.Class') as mock: # замена объекта
    pass
```

Вопрос 7: Как тестировать асинхронные функции в Python?

Ответ:

Использовать библиотеку **pytest-asyncio** и декоратор `@pytest.mark.asyncio`:

```
python
@pytest.mark.asyncio
async def test_async_func():
    result = await async_function()
    assert result == expected
```

Вопрос 8: Что такое «фабрика фикстур» и зачем она нужна?

Ответ:

Фабрика фикстур — это функция, которая возвращает другую функцию для создания фикстур с параметрами. Нужна для:

```
python
@pytest.fixture
def make_user():
    def _make_user(name, age):
        return User(name=name, age=age)
    return _make_user
```

```
# Использование
def test_user(make_user):
    user = make_user("John", 25)
```

Вопрос 9: Как проверить, что функция вызвалась N раз с определенными аргументами?

Ответ:

Использовать методы мока:

```
python
mock.func.assert_called_with(arg)      # вызвана 1 раз с аргументом
mock.func.assert_called_once_with(arg) # вызвана ровно 1 раз
mock.func.assert_has_calls([call(1), call(2)]) # список вызовов
assert mock.func.call_count == 3          # количество вызовов
```

Вопрос 10: Что такое coverage.py и какой порог считается хорошим?

Ответ:

coverage.py — инструмент для измерения покрытия кода тестами

Показывает, какие строки кода были выполнены во время тестов

Хороший порог: 80%+ для проектов, 90%+ для критичных компонентов

Для ответственных систем стремятся к 100%

Вывод: В ходе выполнения практической работы реализован Вариант 2 — Модульное тестирование с моками для сервиса PaymentService.

Разработано 10 тестов, проверяющих логику повторных попыток при TimeoutError с использованием pytest и pytest-mock. Ключевой тест test_retry_logic_on_timeout подтверждает: при ошибке выполняется 3 попытки, затем выбрасывается PaymentFailedException, количество вызовов и аргументы проверяются через call_count и assert_called_with().